

Plan del Proyecto

Instituto Tecnológico de Costa Rica

Escuela de Ingeniería en Computación

Curso: Aseguramiento de la calidad del Software

Periodo: I Semestre 2026

Estudiantes:

Angie Aguilar Alemán

Jonathan Sancho Loaiza

Jeremy Montero Araya

Dereck Cuadra Gonzalez

Docente: Prof. Marcos Rodriguez



Tabla de Contenidos

Índice de Tablas.....	
Introducción.....	
Especificaciones del software.....	
Versiones y proveedores de la aplicación.....	
Hardware utilizado en el entorno de ejecución de la aplicación.....	
Descripción de requerimientos o historias de usuario.....	
Requerimientos del sistema.....	
Requerimientos para pruebas.....	
Problema.....	
Objetivos, metas e indicadores.....	
Objetivo General.....	
Objetivos Específicos.....	
Metas.....	
Indicadores.....	
Recursos Disponibles.....	
Cronograma de actividades.....	
Bibliografía.....	

Índice de Tablas

Tabla 1: Versiones y proveedores de la aplicación	5
Tabla 2: Hardware utilizado en el entorno de ejecución de la aplicación	6

Introducción

Hoy en día, muchas empresas y organizaciones necesitan sistemas digitales para gestionar sus operaciones internas, como el registro de pedidos, el control de usuarios y la protección de la información que manejan. Cuando estos sistemas no funcionan correctamente o no cuentan con las medidas de seguridad adecuadas, pueden ocurrir problemas graves: desde la pérdida de información importante hasta el acceso no autorizado de personas ajenas al sistema.

En este contexto, se identificó una aplicación web de gestión de pedidos que permite a los usuarios registrarse, iniciar sesión y crear pedidos de forma digital. El sistema cuenta con dos tipos de usuarios: los usuarios regulares, que pueden gestionar sus propios pedidos, y los administradores, que tienen acceso completo para supervisar todos los pedidos y usuarios registrados. Para proteger el acceso, el sistema utiliza un mecanismo de autenticación basado en tokens, lo que significa que cada vez que un usuario inicia sesión, el sistema le genera una credencial temporal que le permite operar de forma segura.

Se seleccionó este proyecto porque, a pesar de que el sistema se encuentra funcional, se detectaron áreas de oportunidad relacionadas con la calidad del software y la seguridad de la información. Al tratarse de un sistema que maneja datos de usuarios y controla el acceso mediante roles y credenciales, cualquier falla o vulnerabilidad podría comprometer tanto la operación como la confianza de quienes lo utilizan. Por esta razón, resulta un caso de estudio ideal para aplicar prácticas de aseguramiento de calidad, realizar pruebas estructuradas y proponer mejoras concretas.

El objetivo de este documento es presentar de manera clara el plan de trabajo que guiará las actividades del proyecto. En él se describen las características técnicas del sistema, las funcionalidades que serán evaluadas, el problema central identificado junto con sus causas y efectos, así como los objetivos, metas e indicadores que permitirán medir el avance y el impacto de las mejoras realizadas.

Dado que el sistema involucra el manejo de credenciales, tokens de sesión y distintos niveles de acceso, resulta especialmente relevante aplicar técnicas de verificación que abarquen tanto el comportamiento funcional como los aspectos de seguridad. Un enfoque de pruebas bien definido no solo permite detectar errores antes de que generen consecuencias, sino que también establece un marco de referencia para futuras mejoras sobre la base de código existente.

A lo largo del documento se incluyen también los recursos con los que cuenta el equipo para llevar a cabo las pruebas, la planificación temporal de las actividades y las fuentes consultadas para sustentar las decisiones tomadas durante el proyecto.

Especificaciones del software

El presente documento describe las tecnologías de software y el hardware utilizados en el entorno de ejecución de la aplicación. La aplicación se encuentra encapsulada por medio de contenedores Docker, lo que permite desplegar todos sus componentes (frontend, backend y base de datos) de forma aislada y reproducible dentro de un mismo equipo. En la tabla de versiones, la columna "Tecnología / Componente" indica el nombre de la herramienta o librería utilizada junto con una breve descripción de su función, mientras que la columna "Proveedor y Versión" especifica quién desarrolla dicha tecnología y la versión exacta empleada en el proyecto.

Versiones y proveedores de la aplicación

Tabla 1: Versiones y proveedores de la aplicación

Tecnología / Componente	Proveedor y Versión
Docker Plataforma de contenedores que encapsula la aplicación y sus dependencias en ambientes aislados y reproducibles.	Proveedor: Docker, Inc. Versión: Dependiente del entorno de ejecución
Node.js Entorno de ejecución de JavaScript del lado del servidor, utilizado para el desarrollo del frontend.	Proveedor: OpenJS Foundation Versión: 18.0.0
React Biblioteca de JavaScript para la construcción de interfaces de usuario basadas en componentes.	Proveedor: Meta Platforms, Inc. Versión: 18.3.1
React Router DOM Biblioteca de enrutamiento para aplicaciones React en el navegador.	Proveedor: Remix Software Versión: 7.12.0
Axios Cliente HTTP basado en promesas para realizar peticiones desde el frontend hacia el backend.	Proveedor: Axios Project Versión: 1.13.12
Java Lenguaje de programación orientado a objetos utilizado para el desarrollo del backend.	Proveedor: Oracle Corporation Versión: 25.0.2
Spring Boot Framework de Java que simplifica la creación de aplicaciones backend con configuración mínima.	Proveedor: VMware / Broadcom (Spring) Versión: 4.0.1
JSON Web Token (JWT) Biblioteca para la generación y validación de tokens JWT, utilizada en la autenticación.	Proveedor: JWT Project Versión: 0.13.0
SpringDoc OpenAPI	Proveedor: SpringDoc

Herramienta que genera documentación interactiva de la API REST de forma automática.	Versión: 3.0.1
PostgreSQL Sistema gestor de bases de datos relacional. Se ejecuta como imagen Docker oficial (postgres:18.0.0).	Proveedor: PostgreSQL Global Development Group Versión: 18.0.0

Hardware utilizado en el entorno de ejecución de la aplicación

Todo el software descrito en este documento (frontend, backend y base de datos) se ejecuta en un solo equipo, desplegado mediante contenedores Docker. A continuación se detallan las especificaciones del equipo utilizado.

Tabla 2: Hardware utilizado en el entorno de ejecución de la aplicación

Componente	Especificación
Equipo	ASUS TUF Gaming A15 FA506NC
Procesador	AMD Ryzen 5 7535HS
Memoria RAM	32 GB DDR5 — 4800 MHz
Tarjeta gráfica	NVIDIA GeForce RTX 3050 Mobile
Almacenamiento (1)	NVMe SSD — Patriot M.2 P300 — 512 GB
Almacenamiento (2)	NVMe SSD — Micron MTFDKBA512QGN — 512 GB
Red cableada	Realtek RTL8111 — Gigabit Ethernet
Red inalámbrica	Realtek RTL8852BE — IEEE 802.11ax (Wi-Fi 6)

Descripción de requerimientos o historias de usuario

US-AUTH-001 · Registro de usuario	
Historia de Usuario	Como usuario nuevo quiero registrarme en el sistema para poder acceder a la plataforma y utilizar sus funcionalidades.
Criterios de Aceptación	
Escenario 1: Registro exitoso	
Given	El usuario se encuentra en la página de registro
When	Ingresa sus datos correctamente y presiona "Registrarse"
Then	El sistema crea la cuenta y redirige al usuario al dashboard
Escenario 2: Datos incompletos	
Given	El usuario se encuentra en el formulario de registro
When	Intenta registrarse con campos vacíos
Then	El sistema muestra un mensaje indicando que debe completar los campos requeridos

US-AUTH-002 · Inicio de sesión	
Historia de Usuario	Como usuario registrado quiero iniciar sesión en el sistema para acceder a mi cuenta y a las funcionalidades disponibles.
Criterios de Aceptación	
Escenario 1: Inicio de sesión exitoso	
Given	El usuario tiene una cuenta registrada
When	Ingresa sus credenciales correctamente
Then	El sistema lo redirige al dashboard
Escenario 2: Credenciales incorrectas	
Given	El usuario intenta iniciar sesión
When	Ingresa credenciales incorrectas
Then	El sistema muestra un mensaje de error

US-DASH-003 · Visualización del Dashboard	
Historia de Usuario	Como usuario autenticado quiero visualizar el dashboard del sistema para conocer la cantidad de usuarios y órdenes registradas.
Criterios de Aceptación	
Escenario 1: Mostrar estadísticas	
Given	El usuario ha iniciado sesión
When	Accede al dashboard
Then	El sistema muestra el total de usuarios registrados

And	Muestra el total de órdenes registradas
-----	---

US-ORD-004 · Crear orden	
Historia de Usuario	Como usuario autenticado quiero crear una nueva orden para registrar información dentro del sistema.
Criterios de Aceptación	
Escenario 1: Creación exitosa de orden	
Given	El usuario está en la sección de órdenes
When	Ingresa la descripción de la orden
And	Presiona el botón de guardar
Then	El sistema registra la nueva orden correctamente
Escenario 2: Campos vacíos	
Given	El usuario intenta crear una orden
When	Deja campos obligatorios vacíos
Then	El sistema muestra un mensaje de error

US-ORD-005 · Visualizar órdenes	
Historia de Usuario	Como usuario autenticado quiero visualizar las órdenes registradas para consultar la información existente en el sistema.
Criterios de Aceptación	
Escenario 1: Mostrar lista de órdenes	

Given	Existen órdenes registradas
When	El usuario accede a la sección de órdenes
Then	El sistema muestra la lista de órdenes existentes

US-ADM-006 · Buscar usuarios	
Historia de Usuario	Como administrador quiero buscar usuarios por nombre para encontrar rápidamente un usuario en el sistema.
Criterios de Aceptación	
Escenario 1: Búsqueda exitosa	
Given	El administrador está en el panel de gestión de usuarios
When	Ingresa un nombre en el buscador
Then	El sistema muestra los usuarios que coinciden con la búsqueda

US-ADM-007 · Eliminar usuario	
Historia de Usuario	Como administrador quiero eliminar usuarios del sistema para mantener la base de datos actualizada.
Criterios de Aceptación	
Escenario 1: Eliminación de usuario	
Given	El administrador visualiza la lista de usuarios
When	Selecciona la opción eliminar
Then	El sistema elimina el usuario del sistema

US-ADM-008 · Buscar órdenes	
Historia de Usuario	Como administrador quiero buscar órdenes por descripción para localizar órdenes específicas en el sistema.
Criterios de Aceptación	
Escenario 1: Búsqueda de orden	
Given	El administrador está en la gestión de órdenes
When	Escribe una descripción en el buscador
Then	El sistema muestra las órdenes relacionadas con la búsqueda

US-ADM-009 · Eliminar órdenes	
Historia de Usuario	Como administrador quiero eliminar órdenes del sistema para mantener organizada la información.
Criterios de Aceptación	
Escenario 1: Eliminación de orden	
Given	El administrador visualiza la lista de órdenes
When	Selecciona la opción eliminar
Then	El sistema elimina la orden del sistema

US-AUTH-010 · Cerrar sesión	
Historia de Usuario	Como usuario autenticado quiero cerrar sesión en el sistema para proteger el acceso a mi cuenta .
Criterios de Aceptación	
Escenario 1: Cierre de sesión	
Given	El usuario está dentro del sistema
When	Presiona la opción "Cerrar sesión"
Then	El sistema finaliza la sesión del usuario
And	Lo redirige a la página de login

Problema

Situación actual del sistema

La aplicación evaluada es un sistema web de gestión de pedidos construido con Spring Boot en el backend y React en el frontend, con PostgreSQL como motor de base de datos. El sistema distingue entre dos perfiles de acceso: usuarios regulares, que pueden crear y consultar sus pedidos, y administradores, que disponen de permisos ampliados para gestionar usuarios y órdenes de todo el sistema. La autenticación se resuelve mediante JSON Web Tokens (JWT), generados al momento del inicio de sesión y almacenados en el navegador del cliente. Aunque la aplicación cumple con su propósito funcional básico, una inspección inicial del código y de su configuración reveló debilidades importantes en materia de calidad y seguridad. No se encontraron suites de prueba activas ni en el backend ni en el frontend, y varias decisiones de configuración presentan riesgos que podrían pasar inadvertidos sin un proceso formal de revisión.

Descripción del problema

El proyecto carece de un mecanismo de verificación que permita confirmar, de forma repetible y documentada, que las funcionalidades del sistema operan según lo esperado y que las configuraciones de seguridad son adecuadas. Esta ausencia de control de calidad abarca tanto la capa de servidor como la de cliente, y se extiende a los archivos de configuración del entorno de despliegue.

Origen del problema

A partir del análisis exploratorio del repositorio, se identificaron las siguientes situaciones que dan origen al problema:

1. El backend no cuenta con pruebas unitarias ni de integración. Las dependencias de testing (JUnit, Mockito) están declaradas en el proyecto pero no se utilizan en ningún caso de prueba funcional.
2. No existen pruebas que validen la comunicación entre el frontend en React y la API REST del backend, dejando sin cobertura los flujos completos del usuario.
3. Las credenciales de conexión a PostgreSQL aparecen escritas directamente en los archivos de propiedades del servidor, sin hacer uso de variables de entorno o mecanismos de gestión de secretos.
4. El archivo docker-compose emplea contraseñas triviales e idénticas al nombre de usuario para el servicio de base de datos.
5. El servidor acepta peticiones desde cualquier origen, con cualquier método y cabecera HTTP, debido a una política CORS sin restricciones.

6. La comprobación de vigencia del token JWT ocurre exclusivamente en código JavaScript del cliente, sin que el servidor valide por su cuenta si el token ha expirado. Además, el token se guarda en localStorage, exponiéndolo a posibles ataques de tipo XSS.

Consecuencias identificadas

De no abordarse las situaciones descritas, el sistema queda expuesto a los siguientes escenarios:

7. Errores en flujos clave (registro, login, creación de pedidos) podrían llegar a producción sin ser detectados, afectando directamente la experiencia de quienes usan la plataforma.
8. Un atacante podría aprovechar la política CORS abierta o la validación deficiente de tokens para realizar peticiones no autorizadas a la API, accediendo o modificando información restringida.
9. Las credenciales visibles en el repositorio representan un punto de entrada directo a la base de datos si el código fuente quedara expuesto, comprometiendo todos los registros almacenados.
10. La corrección de defectos descubiertos en etapas avanzadas del desarrollo implica un esfuerzo considerablemente mayor que si se detectaran mediante pruebas tempranas y sistemáticas.
11. Sin métricas de calidad definidas, no existe forma objetiva de determinar en qué estado se encuentra el software ni de medir el progreso de las mejoras aplicadas.

Ante este panorama, el presente proyecto se propone implementar un proceso de aseguramiento de calidad que cubra los aspectos funcionales y de seguridad del sistema, permitiendo documentar hallazgos, corregir deficiencias y establecer una base sólida para el mantenimiento futuro de la aplicación.

Objetivos, metas e indicadores

Objetivo General

Comprobar la estabilidad del software mediante la implementación de estándares técnicos y metodologías de aseguramiento de la calidad reconocidas por la industria.

Objetivos Específicos

1. Evaluar el funcionamiento de las funcionalidades principales mediante la ejecución de pruebas funcionales basadas en historias de usuario.

2. Inspeccionar el código fuente para identificar si existen datos sensibles hardcodeados en lugar de variables de entorno.
3. Comprobar el manejo de tokens de usuario para determinar si la validación de expiración se realiza en el servidor y no únicamente en el cliente.

Metas

1. Ejecutar el 100% de los casos de prueba definidos para los flujos críticos y documentar los resultados obtenidos.
2. Inspeccionar los archivos de configuración en busca de credenciales expuestas.
3. Verificar que el servidor rechace el acceso con tokens de usuario expirados.

Indicadores

12. Porcentaje de casos de prueba ejecutados sobre el total de casos definidos para los flujos críticos.
13. Cantidad de archivos de configuración analizados y hallazgos de datos sensibles reportados.
14. Porcentaje de validaciones de token realizadas correctamente por el servidor en escenarios de expiración.

Recursos Disponibles

Para llevar a cabo las actividades de prueba sobre el sistema, el equipo cuenta con los siguientes recursos técnicos y de información.

Infraestructura y entorno de trabajo

Cada miembro del equipo ejecuta el sistema completo en su propia máquina mediante Docker Compose, lo que levanta de forma simultánea el backend (Spring Boot), el frontend (React) y la base de datos (PostgreSQL) dentro de contenedores aislados. Esta configuración garantiza que todos trabajan sobre un entorno idéntico, elimina problemas de compatibilidad entre sistemas operativos y permite ejecutar pruebas sin depender de un servidor externo ni afectar datos reales.

Datos y cuentas de prueba

La base de datos local se alimenta con registros ficticios que replican situaciones de uso reales: usuarios con rol de administrador y con rol regular, pedidos en distintos estados y combinaciones de datos que permiten ejercitar los diferentes caminos de la aplicación. Asimismo, se dispone de cuentas de prueba predefinidas para cada rol, de modo que los escenarios de autenticación, autorización y gestión de datos se puedan reproducir de forma consistente por cualquier integrante del equipo.

Herramientas para la ejecución de pruebas

En el lado del servidor, el proyecto incluye las dependencias de JUnit 5 y Mockito, lo que permite escribir pruebas unitarias y de integración sobre los controladores, servicios y filtros de seguridad de Spring Boot. Para explorar y validar los endpoints de la API REST, se utiliza la interfaz interactiva generada por SpringDoc OpenAPI (Swagger), complementada con Postman para la creación de colecciones de solicitudes que cubran distintos escenarios. En el frontend, se pueden emplear las utilidades de testing de React para verificar el comportamiento de los componentes de interfaz. Todas estas herramientas se encuentran disponibles en los entornos locales y pueden ejecutarse desde la terminal.

Código fuente y documentación disponible

El equipo tiene acceso completo al repositorio del proyecto, incluyendo el historial de commits, los archivos de configuración y la estructura de ambas capas de la aplicación. La documentación automática de la API generada por SpringDoc permite consultar los endpoints disponibles, sus parámetros y las respuestas esperadas, lo cual sirve como insumo directo para el diseño de los casos de prueba.

Cronograma de actividades

El siguiente cronograma organiza temporalmente las actividades planificadas a lo largo del semestre. Cada semana incluye las tareas a realizar, los responsables y los entregables esperados.

Periodo	Tareas principales	Asignación	Entregable
Sem. 4 (9-15 mar)	Exploración inicial del repositorio y puesta en marcha del entorno local con Docker. Conformación del equipo y asignación de roles. Primera identificación de problemas mediante revisión del código.	Todos	Entorno funcional, equipo conformado.
Sem. 5 (16-22 mar)	Redacción de especificaciones técnicas (software y hardware). Escritura de las historias de usuario con sus criterios de aceptación. Identificación de recursos para	Todos	Borrador de secciones del plan.

	pruebas.		
Sem. 6 (23-29 mar)	Cierre del documento Plan del Proyecto: completar problema ampliado, cronograma, bibliografía. Revisión cruzada entre integrantes.	Todos	Plan del Proyecto entregado (E1).
Sem. 7 (30 mar - 5 abr)	Incorporar observaciones del profesor sobre el plan. Definir la estrategia general de pruebas: niveles, tipos y prioridades.	Todos	Plan ajustado según retroalimentación.
Sem. 8 (6-12 abr)	Diseño detallado de los casos de prueba para los flujos de autenticación y gestión de usuarios. Estructurar el documento del plan de pruebas.	Angie, Dereck	Casos de prueba de auth y usuarios.
Sem. 9 (13-19 abr)	Diseño de casos de prueba para gestión de órdenes, dashboard y seguridad. Seguimiento con el profesor. Preparar datos de prueba en la BD.	Jonathan, Jeremy	Casos restantes. Reunión de avance.
Sem. 10	Consolidar el Plan	Todos	Plan de Pruebas

(20-26 abr)	de Pruebas: integrar todos los casos, redactar conclusiones y revisar coherencia general del documento.		finalizado.
Sem. 11 (27 abr - 3 may)	Entrega formal del Plan de Pruebas. Correcciones menores de última hora si son necesarias.	Todos	Plan de Pruebas entregado (E2).
Sem. 12 (4-10 may)	Configurar JUnit y Mockito en el backend. Escribir las primeras pruebas automatizadas sobre endpoints de autenticación y filtros de seguridad.	Jonathan, Jeremy	Primeros scripts funcionales.
Sem. 13 (11-17 may)	Ampliar cobertura: controladores de órdenes, validaciones de token, escenarios negativos. Registrar defectos encontrados.	Jonathan, Jeremy	Cobertura $\geq$ 80% de casos.
Sem. 14 (18-24 may)	Completar automatización pendiente. Ejecutar suite completa y verificar resultados.	Todos	Suite completa ejecutada.

	Ajustar pruebas que fallen.		
Sem. 15 (25-31 may)	Entrega del Proyecto de Pruebas. Comenzar informe: compilar resultados, generar tablas y gráficos de cobertura.	Todos	Proyecto de Pruebas (E3).
Sem. 16 (1-7 jun)	Redactar informe final: hallazgos, clasificación de defectos, conclusiones y reflexiones del equipo. Preparar presentación.	Angie, Dereck	Informe de pruebas completo.
Sem. 17 (8-14 jun)	Entrega del informe, presentación oral del proyecto y diario reflexivo.	Todos	Informe, presentación, diario (E4, E5, E6).

Bibliografía

- [1] R. S. Pressman y B. R. Maxim, Ingeniería del Software: Un Enfoque Práctico, 8.^a ed. McGraw-Hill, 2015.
- [2] ISTQB, "Certified Tester Foundation Level Syllabus," International Software Testing Qualifications Board, versión 4.0, 2023. [En línea]. Disponible en: <https://www.istqb.org/certifications/certified-tester-foundation-level>.
- [3] OWASP Foundation, "OWASP Top Ten," 2021. [En línea]. Disponible en: <https://owasp.org/www-project-top-ten/>.
- [4] Auth0, "Introduction to JSON Web Tokens," 2024. [En línea]. Disponible en: <https://jwt.io/introduction>.
- [5] Spring, "Spring Security Reference," VMware / Broadcom, 2024. [En línea]. Disponible en: <https://docs.spring.io/spring-security/reference/index.html>.
- [6] M. Fowler, "TestPyramid," martinowler.com, 2012. [En línea]. Disponible en: <https://martinfowler.com/bliki/TestPyramid.html>.